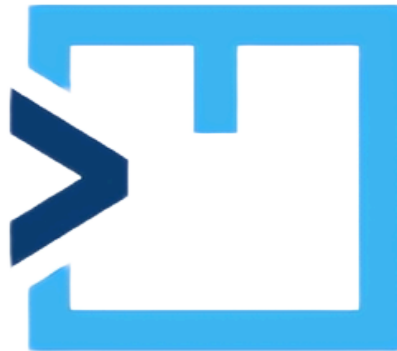


DEUSTOCOMMERCE



1. Descripción General y Alcance del Proyecto	3
1.1. Introducción	3
1.2. Objetivos del Sistema	3
2. Especificación de Requisitos	4
2.1. Requisitos Funcionales	4
2.2. Requisitos No Funcionales	5
3. Arquitectura del Sistema	6
3.1. Diagrama General de la Arquitectura	6
3.2. Descripción de Componentes	6
3.3. Tecnologías y Librerías empleadas	7
4. Diseño de Datos y Persistencia	8
4.1. Modelo de Base de Datos (E - R)	8
4.2. Sistema de Ficheros	9
5. Protocolo de Comunicaciones (Cliente - Servidor)	10
5.1. Descripción del Protocolo	10
5.2. Catálogo de Comandos	10
6. Diseño de Menús y Mapa de Interacción	12
6.1. Interfaz de Administrador	12
6.2. Interfaz de Usuario	12
7. Algoritmia y Lógica de Negocio	13
7.1. Algoritmo de Asignación Logística	13
7.2. Sistema de Reabastecimiento Automático	13
7.3. Gestión de Alertas en Tiempo Real	14
7.4. Cálculo de Balance Financiero	14

1. Descripción General y Alcance del Proyecto

1.1. Introducción

DeustoCommerce es un sistema distribuido que simula el funcionamiento de una plataforma de comercio electrónico mediante línea de comandos (CLI). El proyecto abarca el ciclo completo de venta: desde la entrada de stock en los almacenes hasta la entrega final al cliente.

La arquitectura se compone de tres módulos interconectados:

1. **Administrador (Local):** Gestiona inventario y finanzas.
2. **Servidor (Remoto):** Centraliza la lógica de negocio y las conexiones.
3. **Cliente (Remoto):** Interfaz de compra para el usuario final.

Funcionalmente, destaca por su lógica logística: el sistema administra un inventario descentralizado en múltiples almacenes y optimiza automáticamente los envíos asignando el almacén más cercano al cliente para reducir costes y tiempos.

1.2. Objetivos del Sistema

Objetivos Funcionales (Negocio):

- **Gestión Multi-Almacén:** Control total del inventario distribuido, permitiendo visualizar stock por sede y realizar trasvases de mercancía entre almacenes.
- **Logística Inteligente:** Algoritmo de asignación automática de pedidos que calcula la distancia entre el cliente y los almacenes para optimizar el envío.
- **Automatización:** Sistema de restock automático ante roturas de stock y generación de informes financieros periódicos (gastos vs. beneficios).
- **Experiencia de Usuario:** Flujo completo de registro, login (con 2FA) y seguimiento de pedidos en tiempo real.

Objetivos Técnicos:

- **Conectividad:** Implementación de una arquitectura Cliente-Servidor robusta y concurrente mediante Sockets.
- **Persistencia:** Diseño de un sistema de datos mixto que integra bases de datos relacionales y ficheros de texto.
- **Modularidad:** Desarrollo de código estructurado en C/C++, separando claramente la capa de interfaz, la lógica de negocio y el acceso a datos.

2. Especificación de Requisitos

El sistema **DeustoCommerce** debe cumplir con una serie de requisitos funcionales (qué hace el sistema) y no funcionales (restricciones técnicas y de calidad) para garantizar su operatividad y robustez.

2.1. Requisitos Funcionales

2.1.1. Módulo Administrador (Local)

Aplicación de gestión interna para el control logístico y financiero.

- **Gestión de Inventario Multialmacén:** Visualización del stock global y detallado por almacén. Capacidad para dar de alta/baja productos y modificar precios.
- **Logística Interna:** Funcionalidad para realizar trasvases de mercancía manuales entre almacenes (origen → destino), calculando un coste de transporte simulado.
- **Reabastecimiento (Restock):**
 - **Manual:** Solicitud de stock a proveedores externos para un almacén específico.
 - **Automático:** Proceso que verifica si $\text{stock_actual} < \text{stock_seguridad}$ y genera una orden de compra automática.
- **Finanzas y Reporting:** Generación de balances económicos (Gastos vs. Beneficios). Exportación de datos a ficheros .xlsx (Excel) y generación de gráficos para visualizar tendencias mensuales (libxlsxwriter).

2.1.2. Módulo Servidor (Remoto)

Núcleo del procesamiento que actúa como intermediario entre los datos y los clientes.

- **Gestión de Conexiones:** Escucha en un puerto configurable (Socket) para atender múltiples peticiones de clientes de forma concurrente o secuencial rápida.
- **Lógica de Asignación Logística:** Algoritmo que recibe la ubicación del cliente (coord. X, Y) y asigna el pedido al almacén con stock más cercano, optimizando el tiempo de entrega.
- **Autenticación y Seguridad:** Validación de credenciales contra la base de datos y gestión de sesiones.
- **Logging:** Registro detallado de todas las operaciones (conexiones, errores, transacciones) en ficheros de texto (logs).

2.1.3. Módulo Cliente (Remoto)

Interfaz de consola para la interacción del usuario final con la plataforma.

- **Gestión de Cuenta:** Registro de nuevos usuarios y Login seguro. Implementación de **2FA** (guardaremos un hash de la contraseña creado con la librería libsodium y si la contraseña introducida coincide con el hash guardado se enviará un correo con un código para iniciar sesión).
- **Catálogo y Búsqueda:** Buscador de productos con filtros por nombre, categoría o rango de precios.
- **Proceso de Compra:** Gestión de carrito de la compra (añadir/eliminar items), visualización del coste total y confirmación de pedido.
- **Lista de favoritos:** También se podrán añadir productos a favoritos / deseados y se recibirá un aviso (vía mail) en caso de ponerse en descuento un producto de la lista (Envío del mail a través del proceso del servidor).
- **Seguimiento:** Consulta del estado (simulado) de los pedidos realizados y recibir actualizaciones vía mail (En Proceso → Enviado → Entregado) (Envío del mail a través del proceso del servidor).

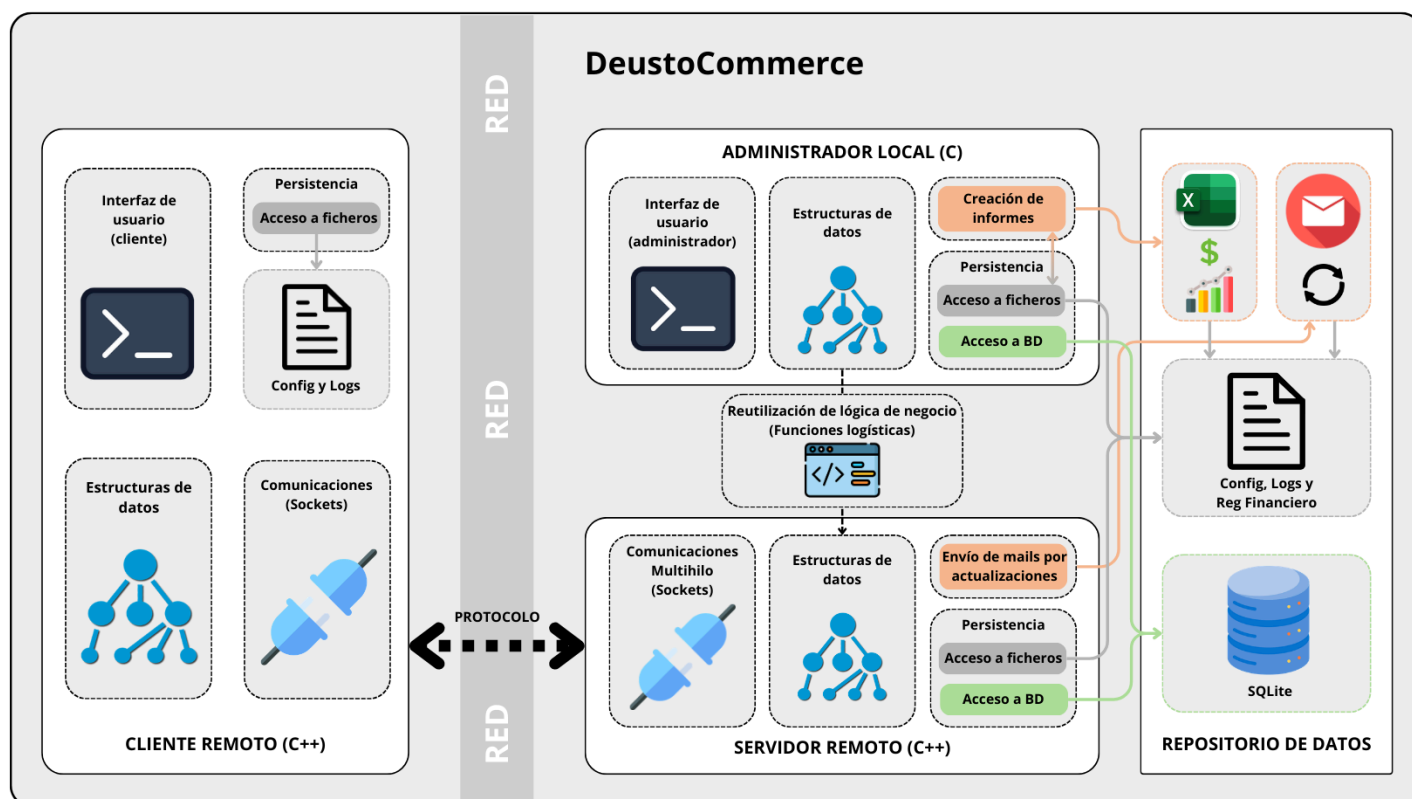
2.2. Requisitos No Funcionales

Restricciones técnicas impuestas por el entorno de desarrollo y la normativa del proyecto.

- **Lenguajes de Programación:**
 - **Módulo Administrador:** Desarrollado en **C** para gestión de bajo nivel y estructuras de datos.
 - **Módulo Servidor y Cliente:** Desarrollados en **C++** para aprovechar la orientación a objetos y diferentes mejoras del lenguaje respecto a C.
- **Comunicaciones:** Uso de **Sockets** TCP/IP asegurando la transmisión fiable de datos.
- **Interfaz:** Diseño en consola (CLI) robusto, con menús claros y validación de entrada de datos para evitar caídas del sistema.
- **Calidad de Código:** Modularización estricta. El Servidor deberá ser capaz de integrar o reutilizar las librerías de lógica de negocio desarrolladas en la Fase 1 (Administrador) garantizando la interoperabilidad C/C++.

3. Arquitectura del Sistema

3.1. Diagrama General de la Arquitectura ([Descargar](#))



3.2. Descripción de Componentes

La arquitectura de **DeustoCommerce** sigue un modelo híbrido Cliente-Servidor con un módulo de administración local. El sistema se estructura en tres componentes principales que comparten una capa común de lógica de negocio.

3.2.1. Admin Local (Fase 1 - C)

Es una aplicación de consola independiente desarrollada en **Lenguaje C**. Su función es permitir la gestión integral del sistema sin necesidad de conexión de red.

- **Acceso a Datos:** Interactúa directamente con la base de datos **SQLite** y el sistema de ficheros para la configuración y logs.
- **Gestión Financiera (Excel):** Módulo específico que procesa los datos indicados en el comando para generar balances, reportes y gráficos exportables en formato .xlsx.
- **Lógica de Negocio:** Integra las funciones logísticas centrales de forma nativa.

3.2.2. Servidor Central (Fase 2 - C / C++)

Actúa como el núcleo de la arquitectura distribuida. Desarrollado en **C++**, su objetivo es atender peticiones remotas y centralizar la lógica en un entorno concurrente.

- **Gestión de Concurrencia:** Implementa un arquitectura **Multihilo** (Thread Pool) que permite la conexión y atención simultánea de múltiples clientes sin bloquear el proceso principal.
- **Reutilización de Código:** Integra la misma librería de lógica de negocio (desarrollada en C) que el Administrador, garantizando coherencia en los cálculos mediante interoperabilidad C/C++.
- **Notificaciones:** Gestiona el envío automático de correos electrónicos (alertas de descuento, confirmaciones, estado de pedido) mediante un cliente SMTP integrado.

3.2.3. Cliente Remoto (Fase 2 - C++)

Interfaz ligera para usuarios externos desarrollada en **C++**.

- **Comunicación:** Se conecta al Servidor mediante Sockets TCP/IP utilizando un protocolo de mensajería serializada definido para el proyecto (gestión de usuario (cambio de nombre, contraseña...), búsqueda y filtrado de productos, gestión del carrito y lista de favoritos).
- **Responsabilidad:** Su función es estrictamente de visualización y petición de datos; no tiene acceso directo a la base de datos ni procesa lógica compleja.

3.3. Tecnologías y Librerías empleadas

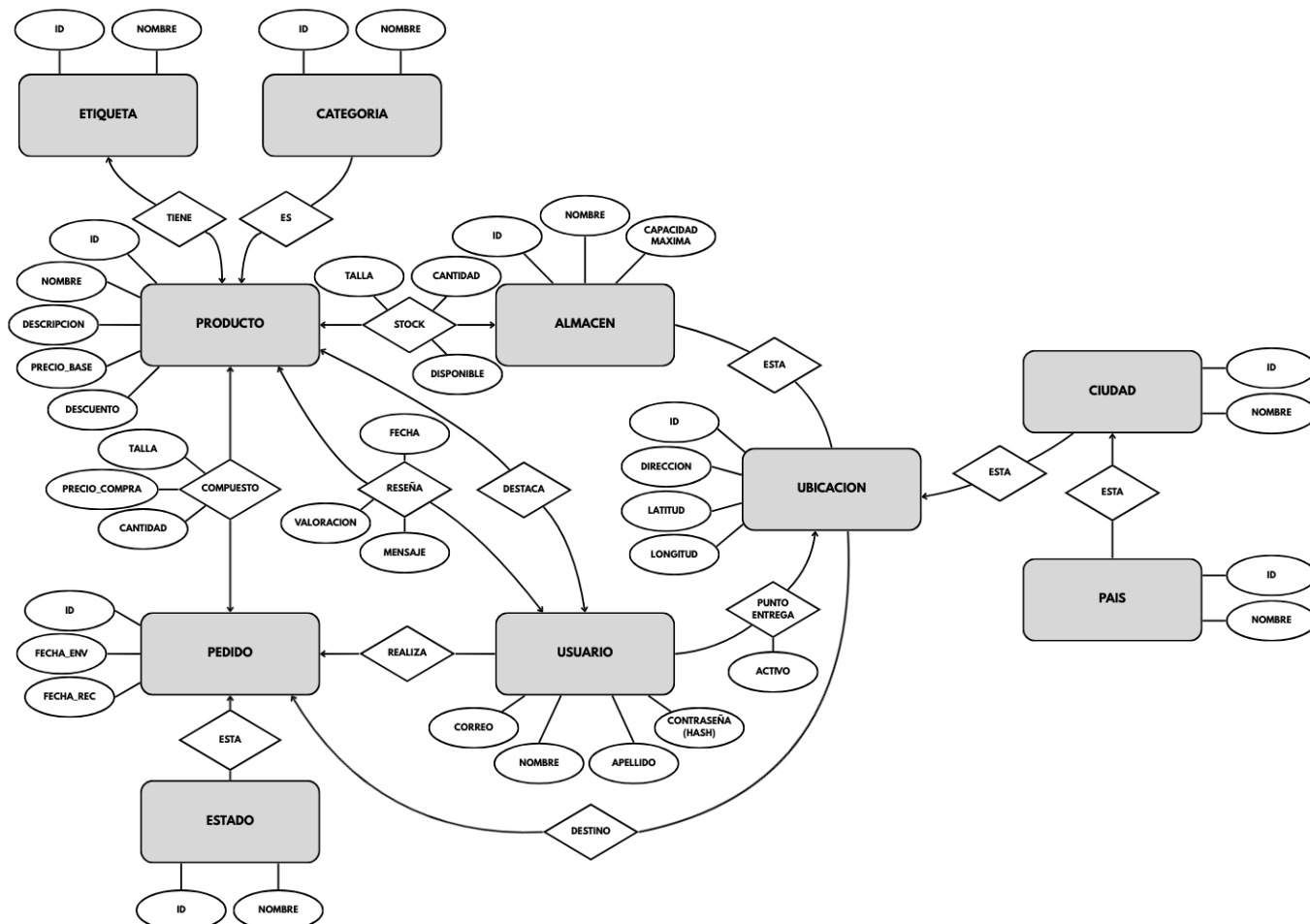
Para el desarrollo del sistema DeustoCommerce se han seleccionado las siguientes tecnologías clave, priorizando la eficiencia en el manejo de datos y la funcionalidad específica de los módulos:

- **Lenguajes C y C++:** Se emplea **C** para el núcleo logístico y el Administrador por eficiencia, y **C++** en el sistema distribuido (Cliente/Servidor) para la gestión de objetos y red.
- **SQLite:** Base de datos que almacena todo el sistema en un fichero local, facilitando la portabilidad sin dependencias externas.
- **Librería de Reportes (libxlsxwriter):** Módulo para la exportación directa de balances y listados financieros a formatos de hoja de cálculo (.xlsx)
- **Cliente SMTP (libcurl):** Librería utilizada para la gestión de comunicaciones de red y el envío automático de correos electrónicos desde el Servidor.
- **Librería de Encriptación (libsodium):** Utilizada para el cifrado y hashing seguro de contraseñas y datos sensibles, garantizando la seguridad en el almacenamiento y autenticación.

4. Diseño de Datos y Persistencia

4.1. Modelo de Base de Datos (E - R)

4.1.1. Diagrama Entidad - Relación ([Descargar](#))



4.1.2. Modelo relacional ([Descargar](#))

Entidades

ETIQUETA (ID_ET, NOM_ET)

CATEGORIA (ID_CAT, NOM_CAT)

ESTADO (ID_ES, NOM_ES)

USUARIO (CORREO, NOM_U, AP_U, CONTR_U)

PAIS (ID_PA, NOM_PA)

CIUDAD (ID_CIU, NOM_CIU, ID_PA)

UBICACION (ID_UB, DIR_UB, LAT_UB, LON_UB, ID_CIU, CORREO, ACTIVO)

PRODUCTO (ID_PR, NOM_PR, DESCRIP_PR, PRECIO_PR, DESCPTO_PR, ID_CAT)

ALMACEN (ID_ALM, NOM_ALM, CAP_MAX, ID_UB)

PEDIDO (ID_PED, F_ENV_PED, F_REC_PED, ID_ES, ID_UB, CORREO)

Relaciones

ETIQUETAS_PRODUCTO (ID_ET, ID_PR)

PRODUCTOS_DESTACADOS (ID_PR, CORREO)

RESENA (ID_PR, CORREO, VALORACION, MENSAJE, FECHA)

STOCK_ALMACEN (ID_PR, ID_ALM, TALLA, DISPONIBLE, CANT)

PRODUCTOS_PEDIDO (ID_PR, ID_PED, TALLA, CANT, PRECIO_COMPRA)

4.2. Sistema de Ficheros

Se utiliza una estructura de ficheros auxiliar para gestionar la configuración, auditoría y carga de datos de forma flexible y complementaria a la BD.

4.2.1. Ficheros de Negocio (Reg Financiero y Alertas)

Se utilizan ficheros (.csv) dedicados para el registro histórico monetario y la gestión de alertas de stock:

- **reg_financiero.csv**: TIMESTAMP;TIPO;CONCEPTO;IMPORTE
- **alertas_stock.csv**: TIMESTAMP;NIVEL;ID_PROD;STOCK_ACTUAL;ID_ALM

4.2.2. Ficheros de Configuración

El uso de ficheros .ini evita el *hardcoding*, permitiendo configurar el despliegue en distintos entornos sin necesidad de recompilar el código.

- **server_config.ini**: Es leído por el Servidor Central al iniciar.
 - **[RED]**: Define el puerto de escucha del Socket (ej. PORT=XX).
 - **[DB]**: Ruta relativa al fichero de la base de datos (ej. DB_PATH=XX.db).
 - **[SMTP]**: Credenciales necesarias para **libcurl** (Servidor SMTP, emisor y contraseña de aplicación) para el envío de notificaciones automáticas.
- **client_config.ini**: Leído por el Cliente Remoto.
 - **[CONEXION]**: Dirección IP y puerto del servidor destino (ej. SERVER_IP=X.X.X.X, PORT=XX).

4.2.3. Ficheros de Logs

Se implementa un registro persistente para la depuración concurrente, siguiendo el formato estandarizado: **[FECHA-HORA] [NIVEL] [HILO] Mensaje**.

- **server.log**: Registra eventos críticos del servidor:
 - Inicio y cierre del servidor.
 - Conexiones y desconexiones de clientes (IPs).
 - Errores de ejecución SQL o excepciones de memoria.
 - Estado de los envíos de correo (éxito/fallo de libcurl).
- **admin.log**: Auditoría de las acciones realizadas desde la consola de administración local.
- **client.log**: Diagnóstico local de la aplicación cliente. Vital para detectar problemas de conectividad y excepciones locales.

5. Protocolo de Comunicaciones (Cliente - Servidor)

5.1. Descripción del Protocolo

El sistema utiliza un protocolo de comunicación sobre Sockets TCP/IP. En DeustoCommerce el **Servidor guía la interacción** para una experiencia de usuario más sencilla.

Destacar también el uso de **flags** que haremos para saber de dónde venimos o qué camino hemos seguido para llegar a cierto comando.

5.2. Catálogo de Comandos

5.2.1. Comandos Comunes

- **EXIT** (Salir de la aplicación)
- **HOME** (Vuelta al Inicio)
- **CATALOGO** (Listado General)

{ VARIACIÓN: SI ES ADMIN }

- **NUEVO_PROD** (Dar de alta un nuevo producto)
 - Nombre → Precio → Almacenes
- **BUSCAR** (Resultados de catálogo filtrados)
 - **F_NOMBRE** (Añadir/Editar filtro por Nombre)
 - **F_CATEGORIA** (Añadir/Editar filtro por Categoría)
 - **F_ETIQUETA** (Añadir/Editar filtro por Etiqueta(s))
 - **BORRAR** (Limpiar filtros)
 - **BUSCAR_YA** (Ejecutar búsqueda)
- **VER_PROD [ID]** (Detalle y Acciones)

{ VARIACIÓN: SI ES CLIENTE } (Casi todas las acciones disponibles solo logueados)

- **COMPRAR ([TALLA]) [CANT]** (Añadir al carrito)
- **FAVORITO** (Añadir / Quitar de lista de deseos)
- **RESENA** (Escribir o leer reseñas)
- **VOLVER** (Volver al listado anterior)

{ VARIACIÓN: SI ES ADMIN }

- **EDITAR** (Modificar campos del producto)
- **ELIMINAR** (Borrar producto (Afecta a todos los almacenes o solo a uno))
- **VOLVER** (Volver al listado anterior)

5.2.2. Comandos Específicos de Cliente

- **REGISTRO** (Crear nuevo usuario)
 - Nombre → Apellido → Correo → Contraseña → 2FA → [OK] // [ERR]
- **LOGIN** (Inicio de sesión)
 - Correo → Contraseña → 2FA → [OK] // [ERR]
- **CARRITO** (Finalizar compra)
 - *PAGAR* (Inicio del proceso de pago)
 - *DIR_GUARDADA* (Utilizar dirección guardada)
 - Elegir → Confirmar → Procesado
 - *DIR_NUEVA* (Introducir nueva dirección)
 - País → Ciudad → Dirección → Confirmar → Procesado
 - *CANCELAR* (Volver a carrito)
 - *VACIAR* (Se vacía el carrito)
 - *SEGUIR* (Se vuelve a la pantalla de buscar)
- **FAVORITOS** (Lista de Favoritos)
- **PEDIDOS** (Historial y Estado)

5.2.3. Comandos Específicos de Administrador

- **ALMACENES** (Visión Global)
 - *NUEVO_ALMACEN* (Inaugurar nueva sede)
 - Nombre → País → Ciudad → Dirección
- **VER_ALMACEN [ID]** (Gestión de Stock)
 - *VER_PROD [ID]* (Descrita arriba → Eliminar prod solo afecta a este almacén)
 - *ADD_STOCK* (Entrada manual de mercancía)
 - *MOVER_STOCK* (Trasvase a otro almacén)
 - *RESTOCK* (Intento de restock adelantado (Llenar hasta el 80%))
 - *ELIMINAR* (Cerrar almacén (Reubicación de stock automática))
- **INFORME** (Generación de Informes)
 - *BALANCE* (Informe Financiero (Ingresos vs Gastos))
 - *DEAD_STOCK* (Detección de productos estancados)
 - *EXPORTAR* (Exportar a .csv)
 - *TOP_VENTAS* (Ranking histórico de ventas)

7. Algoritmia y Lógica de Negocio

Este apartado describe los algoritmos clave que dotan de inteligencia al sistema, optimizando la gestión de recursos y el procesamiento de datos financieros sin depender exclusivamente de las consultas SQL.

7.1. Algoritmo de Asignación Logística

Este módulo es crítico para dos funciones: calcular el envío al cliente y reubicar stock al cerrar un almacén.

- **Cálculo de Distancia:**

Dado que la base de datos almacena las coordenadas exactas de los almacenes y clientes, el sistema utiliza la **Fórmula del Haversine** para determinar la distancia precisa sobre la esfera terrestre

$$d(U_1, U_2) = 2R \arcsin\left(\sqrt{\sin^2\left(\frac{lat_2 - lat_1}{2}\right) + \cos(lat_1) \cos(lat_2) \sin^2\left(\frac{lon_2 - lon_1}{2}\right)}\right)$$

- **Estimación de Tiempo de Entrega:**

Cuando un pedido contiene múltiples productos, es posible que no todos estén en un único almacén. El algoritmo busca los almacenes más cercanos con stock ($A_1, A_2, \dots, A_N$) para cada producto.

El tiempo total estimado (T_{total}) se calcula asumiendo el "peor caso" (el almacén más lejano determina la llegada final del pedido completo) más una penalización por manipulación de carga:

$$T_{total} = \max(d(Cliente, A_i)) \cdot K_{velocidad} + (N_{productos} \cdot K_{manipulación})$$

Donde $K_{\dots}$ son constantes de tiempo definidas en la configuración del servidor.

7.2. Sistema de Reabastecimiento Automático

El sistema abandona el reabastecimiento estático en favor de un algoritmo dinámico que busca la **homogeneidad del inventario**. Este proceso se ejecuta periódicamente (cada semana) y sigue una lógica de llenado inteligente.

- **Condición de Activación:**

El proceso solo se inicia si la ocupación global del almacén cae por debajo del **80% de su capacidad total**.

$$Ocupación_{total} < (Capacidad_{max} \cdot 0,8)$$

- **Cálculo del Nivel Objetivo:**

El sistema calcula hasta dónde debería llegar el stock de cada producto para que todos tengan la misma disponibilidad.

$$Target = \frac{Capacidad_{max} \cdot 0,8}{N_{tiposProductos}}$$

- **Lógica de Reabastecimiento:**

Para cada producto i , se calcula el **Déficit**:

$$Déficit_i = Target - StockActual_i$$

Si $Déficit_i > 0$: Se genera una orden de compra por esa cantidad exacta

Si $Déficit_i \leq 0$: No se pide nada (El producto está por encima de la media)

7.3. Gestión de Alertas en Tiempo Real

El sistema mantiene un fichero vivo (alertas_stock.csv) que se actualiza mediante **triggers en el código**:

- **Al Vender:** Si $Stock < Umbral$, se añade/actualiza la línea en el fichero.
- **Al Reponer:** Si $Stock \geq Umbral$, se borra la línea del fichero.

7.4. Cálculo de Balance Financiero

El módulo financiero integra la librería **libxlswriter** para transformar los registros brutos (.csv) en informes ejecutivos nativos(.xlsx) con visualización gráfica dinámica.

- **Motor de generación**

La lógica central reside en la función `generar_hoja_balance(f_ini, f_fin, intervalo)`, reutilizada tanto por los comandos manuales como por los procesos automáticos.

- **Procesamiento de Datos:** El algoritmo lee el registro histórico y realiza una **agregación temporal** de los ingresos y gastos según el intervalo solicitado.
- **Renderizado Gráfico:** Se mostrará la evolución del Beneficio Neto (Gráfico de Líneas) y los Gastos por Categoría (Gráfico Circular) con ayuda de la librería.

- **Automatización del Balance Anual**

El sistema mantiene un libro contable vivo (Anuario_Financiero_202X.xlsx).

- **Actualización Mensual:** Al final de cada mes, se crea una nueva pestaña con datos del mismo con intervalos de 3 días para garantizar la legibilidad.
- **Consolidación Anual:** Al final del año, se crea una última pestaña que consolida los datos de las pestañas de los meses.